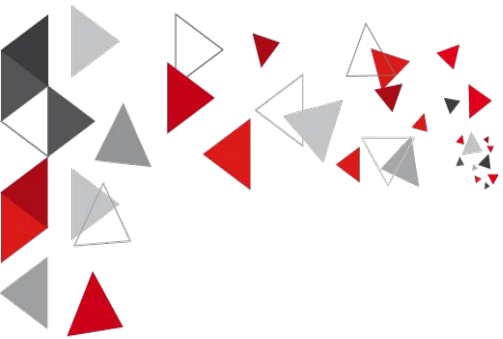


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

Chapitre 6: Polymorphisme

Objectifs du chapitre



- Redéfinition de méthodes dans les sous-classes à travers le polymorphisme
- Comprendre le sur-classement et substitution.
- Découvrir le transtypage (conversion de type ou cast en anglais)



Le polymorphisme consiste à utiliser une méthode

ou un attribut selon plusieurs manières, en fonction du besoin.

Le mot polymorphisme est apparu dans la Grèce antique, Il signifie quelque chose qui peut prendre plusieurs formes.

En Java, le polymorphisme est principalement réalisé grâce à deux mécanismes : l'héritage et l'interface.

Polymorphisme : Définition



Comme vu dans les chapitres précédents, l'héritage permet à une classe d'hériter les attributs et les méthodes d'une autre classe.

Cette sous-classe peut alors redéfinir les méthodes héritées pour leur donner une nouvelle implémentation.

Le polymorphisme par héritage permet à un objet de la sous-classe d'être utilisé comme un objet de la classe parent, ce qui permet une plus grande flexibilité dans l'utilisation des objets.

Polymorphisme : exemple

```java
class Animal {

    public void talk() {
        System.out.println("Je suis un
animal !");
    }
}
```

```java
class Cat extends Animal {

    @Override
    public void talk() {
        System.out.println("Meow !");
    }

    public void scratch() {
        System.out.println("je griffe !");
    }
}
```




```java
class Dog extends Animal {

    @Override
    public void talk() {
        System.out.println("Ouaf !");
    }

    public void attack() {
        System.out.println("j'attaque !");
    }
}
```

Polymorphisme : exemple

```
public class AnimalTester {

    public static void main(String[] args) {
        Dog monChien = new Dog();
        monChien.talk();
        monChien.attack();

        Cat monChat = new Cat();
        monChat.talk();
        monChat.scratch();

        Animal monAnimal = new Dog(); //Instance de Dog déclarée avec référence Animal
        monAnimal.talk();

        Animal monAnimal2 = new Cat(); //Instance de Cat déclarée avec référence Animal
        monAnimal2.talk();

        monAnimal.attack(); //cause une erreur de compilation
        monAnimal2.scratch(); //cause une erreur de compilation

    }
}
```

Polymorphisme : exemple



Dans ce cas pour le compilateur considère que la méthode talk() de **monChien** est la méthode de la classe **Dog**, puisque la référence de l'objet est **Dog**, aussi la JVM va exécuter la méthode de la classe Chien puisqu'elle a été redéfinie.

Dans la même logique le compilateur considère que la méthode talk() de **monAnimal** est la méthode de la classe **Animal**, puisque la référence de l'objet est **Animal**, mais ici la JVM va exécuter la méthode de la classe Chien quand même puisqu'elle a été redéfinie.

Nb: le choix de la méthode à exécuter (pour les méthodes redéfinies) ne se fait pas statiquement à la compilation mais dynamiquement à l'exécution.

Polymorphisme : exemple

```
public class AnimalTester {

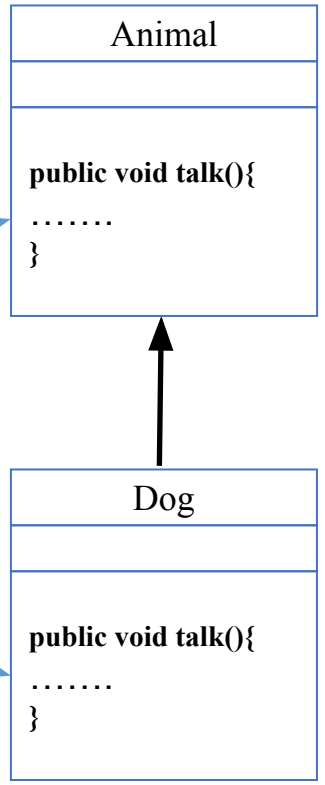
    public static void main(String[] args) {

        Animal monAnimal = new Dog();

        monAnimal.talk();
    }
}
```

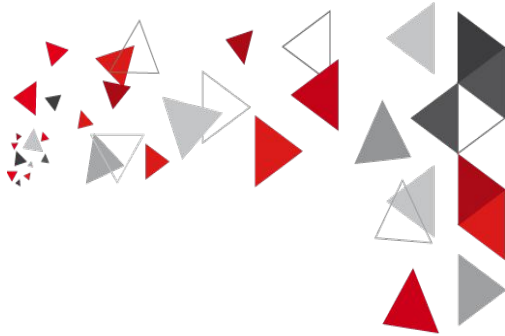






==> Au moment de l'exécution, c'est le niveau le plus bas qui gagne.

Polymorphisme : exemple



```
public class AnimalTester {

    public static void main(String[] args) {

        Animal monAnimal = new Dog();

        monAnimal.attack();
    }
}
```

L'appel de la méthode **monAnimal.attack()** cause une erreur car le compilateur considère que **monAnimal** est un **Animal** et **non pas un Dog**, et donc ne possède pas de méthode nommée attack(), bien qu'il contient une instance de Chien.

Polymorphisme : exemple

l'objet **monAnimal** est dit **surclassé** puisqu'il est déclaré avec la référence de la classe **Animal,** qui ne possède pas la méthode attack().

Il faut donc **substituer monAnimal** par :

((Dog) monAnimal).attack();

La solution est de "convertir" le type de l'objet **monAnimal** à **Dog** pour pouvoir utiliser la méthode **attack()**.

```
public class AnimalTester {

    public static void main(String[] args) {

        Animal monAnimal = new Dog();

        ((Dog) monAnimal).attack();
    }
}
```

Cacher une méthode static



En java, on **ne peut pas** redéfinir une méthode statique mais on peut la cacher si une sous-classe définit une méthode statique avec la même signature qu'une méthode statique de la superclasse.

Ce mécanisme se produit parce que le compilateur choisit quel méthode statique à exécuter au **moment de la compilation.**

Cacher une méthode static : exemple(1/3)

```
public class Animal {

    public static void run() {
      System.out.println("An animal runs");
    }
}

public class Cat extends Animal{

    public void run() {//Erreur de compilation (interdit de redéfinir une méthode statique)
      System.out.println("A cat runs");
    }
}
```

Cacher une méthode static : exemple(2/3)

```
public class Animal {

    public static void run() {
      System.out.println("An animal runs");
    }
}

public class Cat extends Animal{

    public static void run() {
      super.run(); //Erreur de compilation (interdit d'appeler une méthode statique avec super)
      System.out.println("A cat runs");
    }
}
```

Cacher une méthode static : exemple(3/3)



On peut appeler la méthode caché en convertissant l'objet **monChat** en un **Cat** ou en appelant la méthode directement par le nom de la classe.

```
public class AnimalTester {

public static void main(String[] args){
     Animal monAnimal = new Animal();
     Animal monChat = new Cat();

     monAnimal.run(); //An animal runs
     monChat.run(); //An animal runs

     ((Cat) monChat).run(); //A cat runs
     Cat.run(); //A cat runs

    }
}
```

Polymorphisme : Tableaux polymorphiques



Les tableaux polymorphiques en Java sont des tableaux qui peuvent contenir des objets de différentes classes qui héritent d'une même classe parente.

Cela permet de créer des tableaux génériques qui peuvent contenir des objets de différentes classes, mais qui ont des comportements communs définis par leur classe parente.

```
Animal[] animaux = new Animal[2];
animaux[0] = new Dog();
animaux[1] = new Cat();


for (Animal animal : animaux) {
     animal.talk(); // appelle la méthode talk() de chaque type d'objet, soit talk() de
la classe Dog ou talk() de la classe Cat.


}
```

Upcast & Downcast

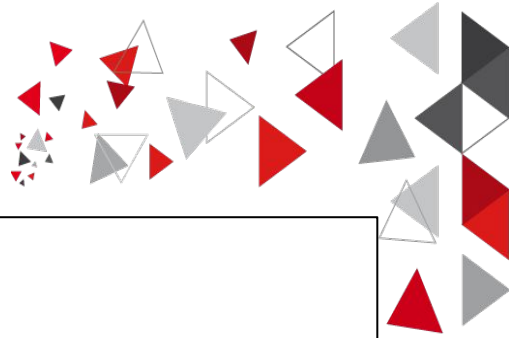


En Java, il existe deux types de **cast** : **l'upcast** et le **downcast**, qui peuvent être **implicites** ou **explicites**.

Un **upcast**, se produit lorsqu'un objet d'une sous-classe est assigné à une variable de sa **super-classe**.

Un **downcast**, se produit lorsqu'un objet d'une superclasse est assigné à une variable de sa **sous-classe**.

Upcast & Downcast



```
public class AnimalTester {

    public static void main(String[] args) {

        Animal monAnimal = new Dog(); //Upcast implicite
        Animal monAnimal2 = (Animal) new Dog(); //Upcast explicite


        Cat monChat = new Animal(); //Downcast implicite
        Dog monChien = (Dog) new Animal(); //Downcast explicite

    }
}
```

Upcast



La conversion vers un type parent **(Upcast implicite et explicite)** est toujours acceptée par le compilateur et elle ne posera aucun problème à l'exécution du programme.

```
public class AnimalTester {

    public static void main(String[] args) {

        Animal monAnimal = new Dog();
        Animal monAnimal2 = (Animal) new Dog(); (Animal) est optionnel
    }
}
```

Downcast

Le **Downcast implicite** est interdit en Java. Le downcast doit être explicite pour éviter les erreurs de compilation.

```
public class AnimalTester {

    public static void main(String[] args) {

        Animal monAnimal = new Dog();
        Dog monChien = monAnimal; //génére une erreur de compilation
     }
}
```

Downcast

Le **Downcast explicite** est toujours acceptée par le compilateur. Cependant, le downcast explicite peut échouer **à l'exécution** s'il est effectué sur un objet qui n'est pas une instance de la sous-classe cible.

```
public class AnimalTester {

    public static void main(String[] args) {

        Animal animal = new Dog();
        Dog chien = (Dog) animal;
        Wolf loup = (Wolf) animal; //génère une exception à l'exécution

    }
}
```

Downcast

Ici l'objet "**monChien**" peut faire appel à la méthode "**attack()**" car après le **Downcast,** il est vu comme un objet de classe **Dog** et non plus comme un **Animal.**

```
public class AnimalTester {

   public static void main(String[] args) {

      Animal monAnimal = new Dog();
      monAnimal.talk(); // Output: "Ouaf !"

      Dog monChien = (Dog) new Animal(); // Erreur d'exécution
      monChien.talk();
      monChien.attack();

   }
}
```

Downcast



Il est important de noter que le **downcast** peut échouer s'il est tenté avec une superclasse qui n'est pas une sous-classe de la classe cible.

Dans ce cas, une exception **ClassCastException** sera levée à l'exécution, mais le compilateur ne présente aucune erreur de compilation.

```
public class AnimalTester {

   public static void main(String[] args) {

      Animal animal = (Dog) new Animal(); //Erreur d'execution (ClassCastException)

   }
}
```

Transtypage des types primitifs

La conversion entre types primitifs permet de convertir la valeur d'un type en un autre type primitif. Cela se produit le plus souvent avec les types numériques.

```
public class Main{

   public static void main(String[] args) {


     float floatValue = 123.45f;
     int intValue = (int) floatValue;
     System.out.println(intValue); //Output: 123 ⇒ Perte de données


     String stringValue = (String) intValue; //Erreur de compilation (types incompatibles)

     Integer integerValue = null;
     intValue = (Integer) integerValue;
     System.out.println(integerValue); //Erreur à l'exécution (NullPointerException)

     }
}
```

Lorsqu'on fait l'affectation d'un entier long (64 bits) à un entier int (32 bits), ou d'un réel double vers un réel float, on risque de perdre une partie de données.

L’opérateur instanceof

instanceof, est un opérateur qui permet de vérifier si un objet est une instance d'une classe spécifique ou d'une de ses sous-classes.

L'opérateur **instanceof** est utile pour effectuer des opérations différentes en fonction du type d'un objet.

```
public class AnimalTester {

   public static void main(String[] args) {

     for (Animal animal : animaux) {
         if(animal instanceof Dog)
              ((Dog) animal).attack();
         if(animal instanceof Cat)
              ((Cat) animal).scratch();
      }
   }
}
```

Nb: l'opérateur instanceof ne fonctionne qu'avec des objets et des classes, pas avec des types primitifs tels que int ou double.

L’opérateur instanceof



Vous pouvez simplifier davantage ce code en créant une variable locale après l'opérateur instanceof, et le compilateur gère automatiquement le downcast.

```
public class AnimalTester {

   public static void main(String[] args) {

     for (Animal animal : animaux) {
         if(animal instanceof Dog dog)
              dog.attack();
         if(animal instanceof Cat cat)
              cat.scratch();
      }
   }
}
```

La méthode getClass()

la méthode **getClass()** renvoie la classe d'un objet au **moment de l'exécution**.

```
public class AnimalTester {

   public static void main(String[] args) {

     Animal animal = new Animal();
     Animal chien = new Dog();

     System.out.println(animal.getClass() == Animal.class); //true
     System.out.println(chien.getClass() == Animal.class); //false

     System.out.println(chien instanceof Dog); //true
     System.out.println(chien instanceof Animal); //true

   }
}
```

La méthode equals()

La méthode **equals** ,de la classe **Object**, permet de comparer deux objets et de déterminer l'égalité entre ces deux objets.

Par défaut, la méthode retourne vrai s'il s'agit du même objet en mémoire, il faut donc la redéfinir pour comparer les objets en fonction de leur contenu.

On suppose qu'un animal est caractérisé par un nom et un âge.

```
public class AnimalTester {

    public static void main(String[] args) {

      Animal animal1 = new Animal("Snoopy", 2);
      Animal animal2 = new Animal("Snoopy", 2);

      //Avant la redéfinition
      System.out.println(animal1.equals(animal2)); //false

      //Après la redéfinition
      System.out.println(animal1.equals(animal2)); //true
}
```

La méthode equals(): exemple 1

Il existe plusieurs implémentations possible pour la méthode equals.

```
public class Animal {

     @Override
     public boolean equals(Object obj) {
          if (null == obj) return false;
          if (obj instanceof Animal animal){
               return age == animal.age && name.equals(animal.name);
          }

          return false;
     }
}
```

Nb: La méthode equals est redéfinie dans la classe String. Elle permet de comparer 2 chaînes de caractères.

La méthode equals(): exemple 2

Il existe plusieurs implémentations possible pour la méthode equals.

```
public class Animal {

     @Override
     public boolean equals(Object obj) {
          if (this == obj) return true;
          if (null == obj) return false;
          if (obj.getClass() == Animal.class){
               Animal animal=(Animal) obj;
               return age == animal.age && name.equals(animal.name);
          }

          return false;
     }
}
```

Merci pour votre attention

